Gradient Cost Function

Mayo (2017) explains that neural networks learn by modifying their weights after each prediction.

The learning process can be divided into four steps:

- The system compares the actual value with the predicted one
- It calculates the cost function (error)
- The error is propagated back through the network using backpropagation
- Each neuron uses this error to update its weights.

In the exercise, I had to change the number of iterations and the learning rate to observe how the cost function changed.

- Changing the learning rate

  Changing the learning rate means adjusting the size of the steps taken during the optimisation.

  The original learning rate was 0.08. I increased it to 0.2 to observe its effect on the optimisation process.

  This made the optimisation process less stable as the cost function oscillated between high and low values because the algorithm overshot the minimum due to excessively large weight updates.

  I then reduced it to 0.01. In this case, the optimisation process became more stable, and the cost function decreased gradually because the parameter updates were smaller and more controlled.
- Changing the number of iterations

  Changing the number of iterations changes how many times the model updates its weights during gradient descent.

The original number of iterations was 100. I increased it to 1000 to observe the effect on the cost function. The optimisation process remained stable, and the cost function decreased gradually because the model performed more weight updates and had more opportunities to converge toward the minimum.

Then I reduced the number of iterations to 30. The model was still able to significantly reduce the cost function, showing that learning had begun effectively, although more iterations are required to achieve full convergence.

Changing the learning rate and the number of iterations helped me understand how these parameters affected the optimisation process. The learning rate directly influences both the stability and the speed of convergence, while a lower learning rate leads to slower but more stable convergence.

Increasing the number of iterations gives the model more opportunities to update its weights and converge toward the minimum of the cost function.